

Productos punto $U4 \times S8$ con corrección de zero-point en un pipeline RV32

Diseño y evaluación de XQDot4Z

8 de septiembre de 2026

Autoría, afiliación y destino editorial: por acordar con el equipo.

Estado del manuscrito

Documento de trabajo: pregunta, diseño propuesto, protocolo y auditoría inicial. La extensión aún no está implementada. No se han medido sus ciclos, recursos, frecuencia ni energía. Los únicos resultados ejecutados de esta versión corresponden a la auditoría del procesador original.

Resumen

El almacenamiento de pesos en cuatro bits reduce su tamaño, pero su consumo en un procesador escalar exige extraer códigos y preparar la aritmética. Este trabajo propone evaluar XQDot4Z, una operación de cuatro productos entre pesos $U4$ y activaciones $S8$ con corrección de zero-point y resultado $INT32$. La pregunta central es cuándo la fusión aporta una ventaja frente a multiplicación escalar y frente a un producto punto empacado con corrección factorizada y reutilización de la suma de activaciones. Se plantea una derivación de kirky-arqui, con base experimental común, verificación aritmética e integración al pipeline, y evaluación pareada de productos punto y matriz-vector. La fase inicial no requiere FPGA: mide exactitud y ciclos RTL; los resultados físicos se incorporarán solo con un flujo y restricciones comparables. La auditoría preliminar reproduce 26 pruebas históricas satisfactorias y documenta limitaciones adicionales del soporte C y del control de dependencias. Se fijan las condiciones de medición y los criterios que permitirán confirmar o rechazar las hipótesis, sin anticipar una aceleración ni afirmar novedad absoluta.

Palabras clave: RISC-V; inferencia cuantizada; $W4A8$; producto punto; microarquitectura; verificación; evaluación reproducible.

Estructura del artículo

1	Introducción	3
2	Fundamentos y antecedentes	3
3	Pregunta, objetivos e hipótesis	4
4	Plataforma y separación de baselines	6
5	Contrato y diseño propuesto	7
6	Metodología de evaluación	9
7	Evidencia inicial y resultados por completar	11
8	Discusión prevista y amenazas a la validez	12
9	Conclusiones provisionales	13
	Referencias	14
A	Artefacto, hitos y reproducción	15

Cómo evoluciona este documento

El artículo se escribe junto con el experimento. Las secciones de diseño y método se actualizan cuando se cierran decisiones; las de resultados y conclusiones se completan únicamente con evidencia. La revisión bibliográfica actual es inicial, no sistemática ni exhaustiva.

1 Introducción

La inferencia de redes neuronales contiene operaciones que multiplican activaciones por pesos y reducen sus productos. Cuantizar estos tensores permite representar el cálculo mediante enteros y parámetros de escala, pero su utilidad depende tanto del formato como de su implementación [1]. En un core escalar, una palabra de 32 bits puede almacenar ocho pesos de cuatro bits; consumirlos puede exigir máscaras, desplazamientos, corrección de origen y multiplicaciones.

Una extensión que realice parte de ese trabajo puede reducir instrucciones. Sin embargo, añade lógica y condiciona el control del pipeline. La reducción del conteo de instrucciones no garantiza menor tiempo si crecen la latencia o el período de reloj. También existe una alternativa algebraica: corregir el producto punto completo usando una suma de activaciones reutilizable. Esta alternativa impide atribuir automáticamente un beneficio a la fusión del zero-point.

Se propone estudiar XQDot4Z sobre kirky-arqui, un procesador docente con pipeline de cinco etapas y soporte parcial de instrucciones RV32I/C. El objeto experimental es el producto punto y la matriz–vector cuantizados de batch uno. La operación no pretende ejecutar por sí sola una red completa ni resolver todos los costos de un modelo de lenguaje.

Las contribuciones candidatas son: (i) un contrato e implementación trazables; (ii) una comparación que distinga multiplicación, procesamiento empacado y corrección fusionada; y (iii) una caracterización de las condiciones que favorecen o perjudican esa fusión. Esta formulación admite resultados negativos: si la corrección factorizada resulta superior, explicar por qué responderá parte de la pregunta de investigación.

2 Fundamentos y antecedentes

2.1 Contrato de cuantización y factorización

Para una cuantización afín se interpreta un código entero mediante $\tilde{r} = s(q - z)$, con escala positiva s y zero-point entero z [1]. Aquí se eligen pesos U4, $q_w \in [0, 15]$, y activaciones S8, $q_a \in [-128, 127]$, con zero-point de activaciones igual a cero. Para la fila r , grupo g e índices I_g , el subtotal es

$$D_{r,g} = \sum_{i \in I_g} (q_{w,r,i} - z_{w,r,g}) q_{a,i}. \quad (1)$$

Con escalas compartidas dentro de cada grupo, una capa reconstruye

$$\tilde{y}_r = b_r + \sum_g s_{w,r,g} s_{a,g} D_{r,g}. \quad (2)$$

El subtotal entero debe ser exacto; el error de cuantización y el del escalado se evalúan por separado. Los grupos con escalas distintas no se combinan como enteros para aplicar después una escala única arbitraria.

Por distributividad, la ecuación (1) también se calcula como

$$D_{r,g} = P_{r,g} - z_{w,r,g} S_{a,g}, \quad P_{r,g} = \sum_{i \in I_g} q_{w,r,i} q_{a,i}, \quad S_{a,g} = \sum_{i \in I_g} q_{a,i}. \quad (3)$$

Si varias filas consumen el mismo vector, $S_{a,g}$ se reutiliza. El baseline debe aprovechar esa propiedad. No hacerlo mediría también una desventaja evitable de software.

2.2 Antecedentes pertinentes y límite de la comparación

XpulpNN estudia instrucciones SIMD de baja precisión y operaciones que combinan producto punto y carga sobre procesadores RISC-V [2]. Dustin demuestra un contexto de aritmética de precisión mixta en un cluster de varios cores [3]. Estos antecedentes impiden presentar el uso de RISC-V o los productos punto de pocos bits como conceptos inéditos.

QServe conecta cuantización W4A8KV4 con kernels y costos de ejecución para servicio de LLM en GPU [4]. Motiva considerar costos fuera de la multiplicación, pero sus resultados no predicen el rendimiento de un core RV32 docente. No se extrapolarán cifras de GPU, ASIC y FPGA entre plataformas.

Referencia	Relación con el proyecto	Lectura adicional requerida
Jacob et al.	Cuantización y aritmética entera.	Compatibilidad del epílogo y reglas de redondeo.
XpulpNN	SIMD sub-byte y fusión de trabajo.	Semántica, operandos y soporte de correcciones.
Dustin	Precisión mixta en RISC-V.	Diferencias de ISA, microarquitectura y memoria.
QServe	W4A8 y costos de conversión.	Contratos de formatos, sin extrapolar speedups.

La revisión es un punto de partida. Antes de enviar un artículo se ampliará la comparación a operaciones empaçadas y extensiones pertinentes, verificando en las fuentes completas anchuras, signo, acumulación y transporte de metadatos. La contribución definitiva se delimitará con esos antecedentes y los resultados.

3 Pregunta, objetivos e hipótesis

Pregunta principal

¿En qué condiciones una operación escalar de cuatro productos $U4 \times S8$ con corrección de zero-point fusionada reduce el costo de ejecutar productos punto y matriz–vector en un pipeline RV32, frente a multiplicación escalar y producto punto empaçado con corrección factorizada, considerando reutilización de activaciones y costo de suministrar el zero-point?

3.1 Descomposición operacional

RQ1: procesamiento empaçado.

Comparar la propuesta con un kernel escalar optimizado que dispone de MUL y recibe los mismos datos $U4/S8$.

RQ2: corrección fusionada.

Comparar con producto punto $U4 \times S8$ sin corrección, seguido de la ecuación (3), reutilizando S_a . Es la comparación central para aislar el zero-point.

RQ3: condiciones de utilidad.

Variar dimensiones, tamaño de grupo y política de suministro de z , sin mezclar sus regímenes.

RQ4: compromiso físico posterior.

Comparar paralelismo y recursos cuando exista un flujo de síntesis/timing común y reproducible.

En la primera fase, costo significa principalmente ciclos del kernel completo. Tiempo físico, recursos y energía se reportan como magnitudes distintas y solo cuando exista la evidencia correspondiente.

3.2 Objetivos

El objetivo general es determinar el alcance útil y las limitaciones de XQDot4Z sobre una derivación verificable de kirky-arqui. Los objetivos específicos son congelar y auditar la base; verificar el contrato aritmético y su integración; construir baselines comparables; medir kernels reproducibles; y preparar una fase física sin confundirla con simulación RTL.

3.3 Hipótesis de trabajo y posibilidad de refutación

- H1. La propuesta reduce ciclos frente a MUL escalar en las configuraciones primarias. Se reportará cada caso y se conservarán empates y degradaciones.
- H2. Su ventaja frente a corrección factorizada disminuye cuando crece el número de filas que reutiliza S_a . Se examinará la tendencia y sus excepciones, sin imponer monotonicidad a los datos.
- H3. El costo de suministrar zero-points variables puede reducir o eliminar una ventaja observada con valores constantes.

Estas expectativas no son hallazgos. Una instrucción correcta con menor rendimiento también es un resultado interpretable. Cambios posteriores en hipótesis, factores o agregaciones se registrarán con fecha y motivo; el protocolo local no se presenta como una preregistración externa.

4 Plataforma y separación de baselines

4.1 Adopción de kirky-arqui

Se conserva el procesador desarrollado en el curso: su organización de cinco etapas, dos lecturas de registros, forwarding y Fetch mixto constituyen una base comprensible para integrar una operación escalar. No se reemplaza el core por una plataforma externa. Se congela el commit `d2e62fd` y se exportan sus 143 archivos versionados en `baseline/upstream/`.

La base es docente y su ISA es parcial. La auditoría dirigida se presenta en la sección 7; el detalle de módulos, líneas y decisiones está en `docs/CORE_AUDIT.md`. El éxito de los programas históricos no se interpreta como certificación de RV32IC completo.

4.2 Correcciones comunes antes de medir

Las memorias actuales tienen 64 palabras de 32 bits cada una y lectura combinacional. Los registros ID/EX se habilitan permanentemente. La lógica de load-use no distingue todos los campos que realmente son registros fuente. El proyecto experimental requerirá capacidades de memoria suficientes, clasificación de operandos, detección de instrucciones excluidas e instrumentación de retiro. Las variantes multiciclo requerirán además retención y validez.

Estos cambios se aplicarán a una base normalizada común. Si una corrección reduce pausas espurias, su beneficio no debe atribuirse a XQDot4Z. Se revisará el subconjunto necesario para los kernels, sin convertir el soporte íntegro de la ISA en un prerequisite del estudio aritmético.

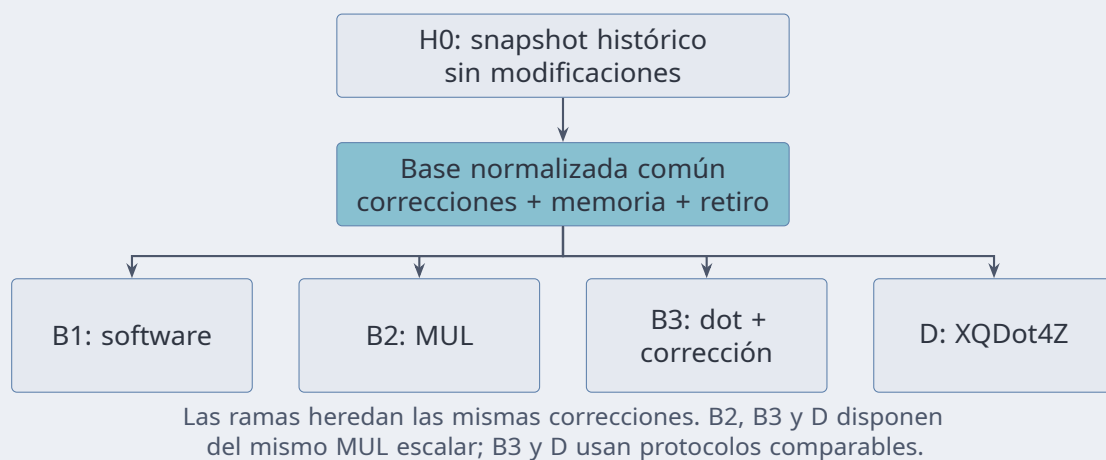


Figura 1: Separación entre procedencia histórica y comparación causal. La base normalizada y sus variantes todavía no están implementadas.

ID	Variante prevista	Comparación interpretada
H0	Original exportado.	Estado del proyecto de curso, no efecto puro del ISA.
B1	Base normalizada; multiplicación en software.	Aporte respecto de un core sin multiplicador.
B2	Base normalizada + MUL escalar.	Beneficio del procesamiento empaado/fusión.
B3	Producto punto $U4 \times S8$ y $P - zS_a$.	Beneficio específico de fusionar la corrección.
D	XQDot4Z.	Costo y utilidad de la propuesta.

MUL por sí sola no constituye la extensión M completa. El estado de soporte de instrucciones y la revisión de la base se fijarán junto con cada resultado.

5 Contrato y diseño propuesto

5.1 Semántica de la unidad aritmética

Un registro de pesos W de 32 bits contiene ocho códigos U4. Un registro A contiene cuatro activaciones S8. El selector $h \in \{0, 1\}$ elige cuatro nibbles bajos o altos de W , mientras $z \in [0, 15]$ es común a los cuatro términos. Para $i = 0, \dots, 3$:

$$q_{w,i} = (W \gg 4(i + 4h)) \& 15, \quad (4)$$

$$b_i = (A \gg 8i) \& 255, \quad (5)$$

$$q_{a,i} = \begin{cases} b_i, & b_i < 128, \\ b_i - 256, & b_i \geq 128, \end{cases} \quad (6)$$

$$d = \sum_{i=0}^3 (q_{w,i} - z)q_{a,i}. \quad (7)$$

La salida es la representación INT32 de d , sin saturación, escalas, bias ni acumulador como tercer operando. El índice cero ocupa los bits bajos. La elección lo/hi es independiente del orden de bytes en memoria; los programas del proyecto usarán little-endian documentado.

Ejemplo de contrato, no medición

Con $W = 0x00004A95$, $A = 0x0102FD04$, $z = 8$ y $h = 0$, los pesos corregidos son $[-3, 1, 2, -4]$ y las activaciones $[4, -3, 2, 1]$. El subtotal es -15 , representado por $0xFFFFFFFF1$. Es un ejemplo matemático de la guía previa, no evidencia de una unidad RTL implementada.

5.2 Rangos y diseño directo

La resta de dos U4 abarca $[-15, 15]$: requiere cinco bits con signo si z es arbitrario. Un producto tiene valor absoluto máximo 1920 y cuatro términos tienen cota 7680. Catorce bits con signo bastan para el subtotal; se permite un árbol conservador de mayor anchura y extensión final a 32 bits.



Ruta directa conceptual; el almacenamiento W4 no obliga a un multiplicador físico de 4 bits.

Figura 2: Operaciones del datapath inicial. Las anchuras útiles deben verificarse antes de optimizar recursos.

Con $z = 8$, el peso corregido sí cabe en S4. Esa especialización y la factorización de la ecuación (3) son alternativas legítimas; no se descartan para favorecer la propuesta.

5.3 Interfaz ISA abierta y control

El nombre XQDot4Z y las variantes lo/hi son provisionales. Una instrucción de 32 bits en espacio personalizado es una opción; el encoding no se congela en esta versión. En particular, z será entrada de cuatro bits de la unidad, pero su transporte desde el ISA sigue abierto.

Un inmediato solo representa un valor presente en el programa. Para zero-points leídos en ejecución se necesitará despacho, especialización previa o una interfaz adicional. Cualquier estado nuevo exige definir hazards, reset y cancelación; su costo no se omite del experimento.

La primera unidad aislada será combinacional. Más adelante se podrán comparar cuatro, dos y una vía. Número de pasos, latencia e intervalo de iniciación son conceptos distintos. La integración deberá capturar operandos resueltos por forwarding, retener una operación pendiente y transferir su resultado una sola vez. Detener F/D/E mientras M/W drenan requiere burbujas o validez equivalente; copiar repetidamente controles no es un protocolo correcto.

Se verificarán $rd = x0$, fuentes/destino solapados, dependencias load-use, consumo inmediato del resultado, stores, instrucciones consecutivas y flush/reset. La corrección del RTL se comprobará contra una referencia entera independiente, no contra la salida aproximada de una red neuronal.

6 Metodología de evaluación

6.1 Verificación antes de rendimiento

Se separan cuatro niveles: referencia matemática, unidad RTL, integración al pipeline y kernel completo. Cada nivel tiene un oráculo y una puerta de aceptación. Para la unidad se usarán extremos, ambas mitades, cancelaciones y vectores aleatorios con semilla; para el pipeline, dependencias y efectos arquitectónicos únicos. Aprobar una batería finita no equivale a una prueba formal de todos los estados.

La regresión del core se amplía antes de medir. El software usa un subconjunto de instrucciones permitido y comprobable. No basta compilar para RV32IC: el compilador puede emitir instrucciones estándar que el core docente no cubre. La primera versión de los kernels se escribirá en ensamblador controlado. Los kernels primarios usarán instrucciones de 32 bits, sin compresión ni relajación automática, con la misma política en todas las variantes. El soporte C del core se conservará y verificará por separado; no se introduce como otro factor de la comparación inicial.

6.2 Configuraciones predefinidas

Factor	Plan inicial, aún no ejecutado
Kernel	Subtotales INT32 por grupo; después, capa con epílogo completo.
Batch	Uno.
Dimensiones	$K \in \{32, 128, 512\}$, $N \in \{1, 4, 16\}$.
Grupo principal	$G = 32$.
Sensibilidad de grupo	$G \in \{8, 32, 128\}$, solo cuando divide K .
Semillas	Diez: 20260908 a 20260917; conservar también tensores y hashes.
Regímenes de z	Constante, especializado y leído en ejecución; resultados separados.
Memoria inicial	Propuesta de 64 KiB IMEM + 64 KiB DMEM, común a variantes.
Microarquitectura	Inicial paralela; dos vías e iterativa en fase posterior.

$N = 1$ constituye el producto punto; $N > 1$ permite reutilización entre filas. Casos con colas, por ejemplo $K \in \{1, 3, 5, 31, 33, 127\}$, se verifican primero como funcionalidad. No se incorporan silenciosamente a un agregado de rendimiento definido para otras configuraciones. La capacidad de memoria propuesta se confirmará al generar los programas; cualquier cambio será común y registrado.

6.3 Política de zero-point y baseline factorizado

ZC: constante por ejecución.

Campañas independientes para $z = 0, 8, 15$. Todas las variantes conocen el mismo valor antes de ensamblar.

ZS: especializado.

Los valores cambian entre fila/grupo, pero se conocen antes de generar código. Se cuenta el código adicional y todas las variantes tienen la misma información para optimizar.

ZR: leído en ejecución.

Los metadatos llegan por memoria. Si D usa inmediatos, se cuenta su despacho; si cambia el ISA para suministrarlos, se identifica como variante adicional. Su implementación

sigue abierta.

B3 calcula $S_{a,g}$ una vez por vector y grupo y lo comparte entre filas; no se fuerza su recálculo para favorecer a D. B3 y D usan el mismo número de vías y protocolos comparables, aunque sus costos lógicos pueden diferir. Los dos disponen de MUL escalar para las operaciones restantes.

Los pesos permanecen empacados en U4 en la comparación principal. Una posible recodificación a S4 cuando $z = 8$ pertenece a otra comparación explícita. La mejora debida a reducir bytes desde FP32 no se atribuye a una instrucción cuando el baseline ya recibe los mismos datos U4.

6.4 Frontera de medición y métricas

La región comienza con la aceptación de la primera instrucción del kernel y termina con el retiro de la última que materializa su salida. El testbench deberá implementar y documentar esos eventos. Se excluyen reset y carga inicial del programa; se incluyen acceso a tensores, bucles, corrección, suma de activaciones, despacho de metadatos y stores de salida.

Se medirá aparte el núcleo aritmético y, cuando exista, la capa con epílogo. El subtotal entero por grupo no se llamará “inferencia completa”. Las magnitudes principales son

$$S_{\text{ciclos}} = \frac{C_B}{C_D}, \quad \text{CPI} = \frac{C}{I_{\text{retiradas}}}, \quad t = \frac{C}{f}. \quad (8)$$

El contador de retiro incluye stores y branches, no solo instrucciones con escritura de registros. Se conservarán conteos absolutos, stalls desglosados, código y tráfico por nivel de memoria. Sin timing sustentado, S_{ciclos} no se presentará como speedup físico medido.

6.5 Análisis y fase física

Se compararán variantes de forma pareada por configuración, semilla y tensores. Los casos con $S < 1$ se publican como degradaciones. Cualquier media geométrica indicará qué cocientes integra; no sustituirá la tabla de casos completos. La repetición de una simulación determinista con idénticos datos no genera por sí sola evidencia estadística adicional. Si se calculan intervalos, se identificará su unidad de muestreo.

Una fase posterior evaluará modelos de memoria con espera y unidades de una, dos y cuatro vías. Síntesis y place-and-route deberán usar dispositivo, herramienta y restricciones documentados. Se distinguirá un reloj común viable de la frecuencia alcanzada por cada diseño. LUT, FF, DSP y BRAM no son una medida directa de energía; esta requiere potencia y tiempo bajo una metodología explícita. La FPGA y su acceso universitario siguen pendientes.

6.6 Reproducibilidad del experimento

Cada ejecución conservará revisión del core y del protocolo, versiones y opciones de herramientas, comando, configuración, semilla, hashes del programa y tensores, salida esperada/obtenida y logs. El esquema inicial vive en `results/schema.json`. No se han creado filas sintéticas de rendimiento. Un fallo se conserva con su estado; una tabla del artículo se generará desde los datos, no desde números transcritos a mano.

7 Evidencia inicial y resultados por completar

7.1 Auditoría ejecutada del core histórico

La evidencia seleccionada corresponde a 20260908T070537358477Z, sobre el commit d2e62fd, usando Icarus Verilog 13.0. El runner verifica hashes de la exportación antes y después, ejecuta sobre una copia temporal y conserva salidas, diagnósticos y un resumen JSON.

Observación reproducida

La batería histórica produjo 26 PASS y 0 FAIL. En ocho sondas dirigidas adicionales, tres controles coinciden con lo esperado; cuatro expansiones C difieren y una sonda reproduce una pausa espuria por un campo que no es un registro fuente.

Sonda de expansión	Entrada	Esperado	Observado
c.li x5,1	4285	00100293	00004285
c.addi4spn x8,sp,4	0040	00410413	00000040
c.addi16sp sp,16	6141	01010113	00010137
c.ebreak	9002	00100073	000000E7

Cuadro 1: Salida del decompresor histórico en sondas unitarias. No son programas completos ejecutados en el pipeline.

Las expansiones esperadas siguen la especificación de instrucciones comprimidas [5]. La sonda de control de dependencias representa lw x5, ... seguida de addi x6,x0,5: los bits 24:20 contienen 5, pero ADDI no lee rs2. La unidad produce StallF=1, aunque la dependencia considerada no existe. Esta prueba observa una señal de control; no mide el efecto temporal sobre un kernel.

El runner histórico utiliza mensajes de éxito y filtra algunas advertencias. Por ello, su resultado se conserva como prueba de humo, complementada con sondas explícitas y futura verificación de retiro. Las limitaciones observadas no impiden estudiar una unidad aislada, pero deben acotarse o resolverse antes de medir programas que dependan de ellas.

7.2 Resultados experimentales aún no disponibles

Resultado	Estado actual	Evidencia requerida
Exactitud de XQDot4Z RTL	Pendiente	Unidad implementada y testbench con oráculo.
Integración al pipeline	Pendiente	Dependencias, cancelación y efectos únicos.
Ciclos B2 frente a D	Pendiente	Kernels y región medida comunes.
Ciclos B3 frente a D	Pendiente	Corrección factorizada con S_a reutilizado.
Sensibilidad N/K/G/z	Pendiente	Campañas completas, incluidas degradaciones.
Recursos y timing	Pendiente	Dispositivo y flujo físico comparables.
Energía y calidad ML	Fuera del mínimo actual	Metodologías y tareas adicionales.

Las celdas pendientes no contienen ceros ni cifras ilustrativas: no hay una medición de aceleración de XQDot4Z en esta versión. La auditoría del core es evidencia habilitante y no confirma H1, H2 ni H3.

8 Discusión prevista y amenazas a la validez

8.1 Validez interna

La amenaza principal es comparar plataformas con correcciones distintas. Se controla derivando variantes de una base común y registrando sus cambios. La segunda es debilitar el baseline: B3 debe reutilizar S_a , disponer de MUL y usar una disposición de datos comparable. El despacho de zero-points, colas y epílogo tampoco puede desaparecer de la región medida.

La base docente requiere ampliar verificación. Un resultado numérico final correcto no detecta necesariamente efectos duplicados, instrucciones ilegales ignoradas o contadores incorrectos. El plan incluye controles de retiro, selección de fuentes y detección de instrucciones fuera del subconjunto. La medición se habilita solo después de cerrar esos riesgos para los kernels.

8.2 Validez de las métricas y alcance externo

Menos instrucciones no implica menos ciclos; menos ciclos no implica menor tiempo si cambia la frecuencia; menos tiempo tampoco prueba menor energía. Se evita usar esas magnitudes como sinónimos. El tiempo que tarda el simulador en el computador anfitrión no es el tiempo del procesador modelado.

Productos punto sintéticos y matrices pequeñas permiten aislar causas, pero no representan por sí solos una aplicación de IA ni un LLM. Tampoco una mejora en memoria combinacional predice automáticamente el resultado con BRAM o cache. Las afirmaciones se limitarán al dominio efectivamente evaluado.

Con $z = 8$, la recodificación a S4 puede ofrecer otra ruta eficiente. Con muchas filas, la amortización de S_a puede favorecer a B3. Con metadatos dinámicos, el despacho puede

consumir los ahorros de D. Estos casos son posibles explicaciones que habrá que contrastar, no conclusiones anticipadas.

8.3 Criterios de discusión de los resultados

Para cada diferencia se buscará un mecanismo: extracción, instrucciones de corrección, carga de operandos, reutilización, stalls, camino crítico o código especializado. Se contrastarán desgloses con el total del kernel. Si no aparece ventaja, se documentará qué costo la elimina y hasta dónde puede generalizarse. No se cambiará a posteriori la pregunta para ocultar una hipótesis refutada.

8.4 Integridad del artefacto y publicación

La exportación histórica no contiene un archivo LICENSE en su raíz. La publicación del derivado requiere resolver permisos, contribuciones y licencia con las personas que participaron en el proyecto de curso. Autoría, afiliación y venue no se asignan en esta versión. Una plantilla editorial específica se adoptará cuando exista una convocatoria apropiada y evidencia suficiente.

9 Conclusiones provisionales

Se ha delimitado un estudio de productos punto $U4 \times S8$ con corrección de zero-point sobre una derivación de kirky-arqui. El protocolo separa el beneficio del procesamiento empacado del aporte específico de fusionar la corrección. También identifica el suministro de metadatos y la reutilización entre filas como factores centrales de la comparación.

La evidencia disponible establece que la batería histórica pasa y que existen limitaciones adicionales fuera de esos programas. Esto justifica conservar el core como base, con auditoría y normalización comunes antes de medir. Todavía no permite concluir que XQDot4Z sea más rápida o eficiente.

El siguiente hito es cerrar el contrato de la unidad, construir un oráculo reutilizable y verificar el RTL aislado. La integración, los kernels y la evaluación física seguirán mediante puertas de aceptación explícitas.

Referencias

- [1] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. arXiv:1712.05877. <https://arxiv.org/abs/1712.05877>, 2017. Fuente primaria; consulta: 8 de septiembre de 2026.
- [2] Angelo Garofalo, Giuseppe Tagliavini, Francesco Conti, Luca Benini, and Davide Rossi. XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes. arXiv:2011.14325. <https://arxiv.org/abs/2011.14325>, 2020. Consulta: 8 de septiembre de 2026.
- [3] Gianmarco Ottavi, Angelo Garofalo, Giuseppe Tagliavini, Francesco Conti, Alfio Di Mauro, Luca Benini, and Davide Rossi. Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode. arXiv:2201.08656v2. <https://arxiv.org/abs/2201.08656v2>, 2023. Versión revisada; publicación relacionada: DOI 10.1109/TCSI.2023.3254810.
- [4] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. arXiv:2405.04532v3. <https://arxiv.org/abs/2405.04532v3>, 2025. Primera versión: 2024; revisión consultada: 1 de mayo de 2025.
- [5] RISC-V International. The RISC-V Instruction Set Manual, Volume I: C extension for compressed instructions, version 2.0. <https://docs.riscv.org/reference/isa/unpriv/c-st-ext.html>, 2026. Biblioteca de especificaciones; fecha de consulta: 8 de septiembre de 2026. El año identifica la consulta, no la introducción de C.

A Artefacto, hitos y reproducción

A.1 Organización

El proyecto vive en `stuff/xqdot4z/`. La guía previa permanece en `stuff/ai_review/`; no se sustituye por este artículo.

Ruta	Responsabilidad
<code>baseline/</code>	Snapshot histórico, commit y manifest de hashes.
<code>audit/</code>	Sondas, logs y evidencia de auditoría.
<code>docs/</code>	Acta, decisiones, auditoría, protocolo y plan editorial.
<code>paper/</code>	Manuscrito, bibliografía y metadatos seleccionados.
<code>model/, rtl/, tests/</code>	Espacios reservados para M1–M3, aún sin extensión.
<code>benchmarks/, results/</code>	Kernels futuros y esquema de resultados.
<code>scripts/</code>	Reproducción de auditoría y comprobación del artefacto.

A.2 Puertas de aceptación

Hito	Estado	Criterio
M0: bases	Preparado	Protocolo, snapshot, auditoría y manuscrito trazables.
M1: contrato	Pendiente	Signo, packing, rangos, selector y casos definidos.
M2: unidad	Pendiente	Cero discrepancias en la batería RTL declarada.
M3: integración	Pendiente	Base común, retiro válido y regresión ampliada.
M4: kernels	Pendiente	B1/B2/B3/D comparables, salidas y métricas verificadas.
M5: evaluación	Pendiente	Todos los casos, análisis y artículo actualizados.
M6: fase física	Pendiente	Herramientas, FPGA y restricciones acordadas.

M1 y M2 no requieren modificar el core. Los ciclos de rendimiento no se comparan antes de validar la instrumentación de M3.

A.3 Comandos y evidencia seleccionada

Desde la raíz del proyecto:

```
make baseline-check
make audit
make check
make paper
```

Cada auditoría crea un directorio nuevo; no sobrescribe evidencias previas. El manuscrito selecciona explícitamente el run en `paper/metadata.tex`. La evidencia inicial está en

`audit/results/20260908T070537358477Z/summary.json`

y la procedencia completa corresponde al commit

`d2e62fd127d85ac33fe54c9b98cffa06970b3fd5`

El comando de auditoría puede terminar correctamente aunque registre las diferencias conocidas: indica reproducción terminada, no conformidad ISA. La comparación futura de la extensión tendrá otro criterio: cero discrepancias respecto al contrato dentro de su batería de aceptación.

El texto se mantendrá en una columna para revisión. El estilo Nord y los diagramas TikZ están separados del contenido y podrán adaptarse a una plantilla editorial. La bibliografía se gestiona con BibTeX, sin descargas al compilar.